

Log Barrier Methods and IPOPT

From KKT to Polynomial-Time Interior Point Algorithms

Dr. Robert Hildebrand

Lecture — April 21, 2026

Grado Department of Industrial & Systems Engineering
Virginia Tech

The Problem We Are Solving

For the rest of this course, the master problem is the **general constrained nonlinear program (NLP)**:

$$\begin{array}{ll}\min_{\mathbf{x} \in \mathbb{R}^n} & f(\mathbf{x}) \\ \text{s.t.} & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p\end{array}$$

Standing assumptions (unless stated otherwise):

- f , g_i , h_j are twice continuously differentiable (C^2).
- A constraint qualification holds at any solution (e.g. LICQ or MFCQ).
- The feasible set has nonempty *strict interior*: $\exists \mathbf{x}$ with $g_i(\mathbf{x}) < 0$ for all i .

Our goal today. We already know *what* a solution looks like — a KKT point. We now want an *algorithm* that actually *finds* one, even when m, n are large.

Convexity of f and g_i is *not* assumed. IPOPT handles nonconvex NLPs — it just loses the global-optimality guarantee in that case.

Where We Are

Already covered: unconstrained methods (steepest descent, Newton, CG, BFGS, L-BFGS), line search, trust regions, KKT conditions, Lagrangian duality.

The gap: we know *when* a point is optimal (KKT), but how do we *find* it for a large constrained problem?

Today: the barrier idea — convert inequalities into a smooth objective penalty, trace the central path, run Newton on a perturbed KKT system. This is exactly what **IPOPT** does.

References for today. Nocedal & Wright Ch. 19; Biegler Ch. 10–11; Wächter's lecture notes (IPOPT creator); Luenberger & Ye Ch. 15.

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

The KKT system is a combinatorial headache

Given $\min f(\mathbf{x})$ s.t. $g_i(\mathbf{x}) \leq 0$, $h_j(\mathbf{x}) = 0$:

$$\nabla f + \sum_i s_i^* \nabla g_i + \sum_j y_j^* \nabla h_j = 0,$$

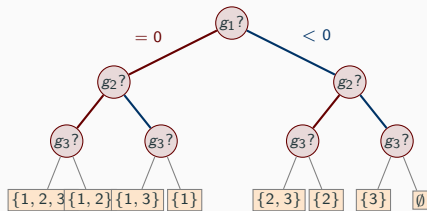
$$h_j = 0, \quad g_i \leq 0, \quad s_i^* \geq 0,$$

$$s_i^* g_i(\mathbf{x}^*) = 0 \quad (\text{complementary slackness}).$$

Either $s_i = 0$ or $g_i(\mathbf{x}) = 0$ for each i . For m inequalities, 2^m active sets.

Active-set methods (SQP, simplex) enumerate cleverly; scale poorly for large m .

Interior point methods take a different route: *smooth out the complementarity*. No enumeration.



$m = 3 \Rightarrow 2^3 = 8$ cases. red = g_i active, blue = g_i slack.

2D KKT in full: $\min x_1 + x_2$ s.t. $x_1^2 + x_2^2 \leq 1$

A single inequality, $g(\mathbf{x}) = x_1^2 + x_2^2 - 1 \leq 0$. KKT conditions write out completely:

$$\begin{aligned} \text{(stat)} \quad \nabla f + s \nabla g &= \begin{pmatrix} 1 \\ 1 \end{pmatrix} + s \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} = \mathbf{0}, & \text{(feas)} \quad x_1^2 + x_2^2 &\leq 1, & \text{(dual feas)} \quad s &\geq 0, \\ & \text{(compl)} \quad s(x_1^2 + x_2^2 - 1) &= 0. \end{aligned}$$

Enumerate the $2^1 = 2$ active sets.

Case 1: $s = 0$ (constraint slack). Stationarity becomes $(1, 1) = \mathbf{0}$. Contradiction — no solution.

Case 2: $s > 0$ (constraint active). Then $x_1^2 + x_2^2 = 1$ and from stationarity $x_1 = x_2 = -1/(2s)$. Plug in: $2/(4s^2) = 1 \Rightarrow s = 1/\sqrt{2}$. So $x_1 = x_2 = -1/\sqrt{2}$.

Unique KKT point: $\mathbf{x}^* = (-1/\sqrt{2}, -1/\sqrt{2})$, $s^* = 1/\sqrt{2}$, $f^* = -\sqrt{2}$. ✓

What you just did by hand. Enumerated $2^m = 2$ cases, solved each system, ruled out the inconsistent one. Easy here; for $m = 30$ it's a billion cases. IPM smooths the complementarity $s \cdot (-g) = \mu$ and lets Newton do it.

The one-dimensional caricature

Consider $\min_x x$ s.t. $x \geq 0$.

KKT: stationarity $1 - s = 0$, feasibility $x \geq 0$, complementarity $sx = 0$. Optimum:

$x^* = 0$, $s^* = 1$.

The barrier idea. Replace the hard constraint $x \geq 0$ with a *penalty* that blows up as $x \rightarrow 0^+$:

$$\min_x x - \mu \log x, \quad \mu > 0.$$

Stationarity: $1 - \mu/x = 0 \Rightarrow x^*(\mu) = \mu$.

- Each $\mu > 0$ gives a *strictly interior* minimizer.
- As $\mu \downarrow 0$, $x^*(\mu) \rightarrow 0 = x^*$.
- Define dual variable $s(\mu) := \mu/x^*(\mu) = 1$. Complementarity $s \cdot x = \mu$ — perturbed, not zero.

Two ideas in one slide. (1) A *smooth* unconstrained problem per μ . (2) Complementarity becomes $s \cdot x = \mu$, tunable via μ .

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

The log barrier function

For inequalities $g_i(\mathbf{x}) \leq 0$, $i = 1, \dots, m$, the **log barrier** is

$$\Phi(\mathbf{x}) = - \sum_{i=1}^m \log(-g_i(\mathbf{x})),$$

defined on the **strict interior** $\{\mathbf{x} : g_i(\mathbf{x}) < 0 \text{ for all } i\}$.

Properties.

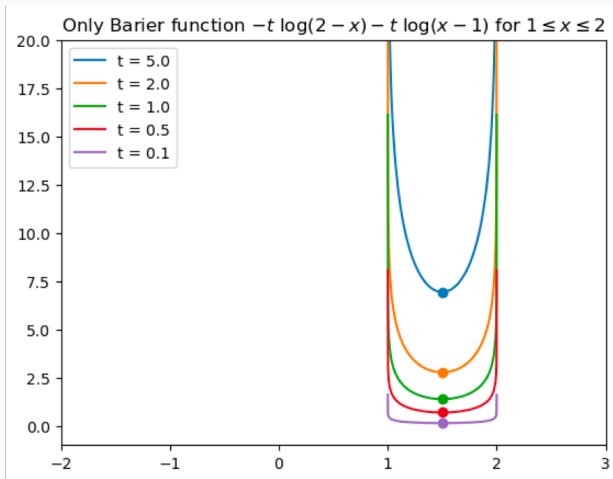
- Φ is smooth on the strict interior.
- $\Phi(\mathbf{x}) \rightarrow +\infty$ as $\mathbf{x}$ approaches the boundary (any $g_i(\mathbf{x}) \rightarrow 0^-$).
- If each g_i is convex, Φ is convex.

Gradient (use chain rule, $\frac{d}{dz} \log z = 1/z$):

$$\nabla \Phi(\mathbf{x}) = \sum_{i=1}^m \frac{\nabla g_i(\mathbf{x})}{-g_i(\mathbf{x})}.$$

The quantity $\frac{1}{-g_i(\mathbf{x})}$ will become the “barrier estimate” of the dual multiplier s_i . Hold that thought.

The log barrier alone: a bathtub on $[1, 2]$

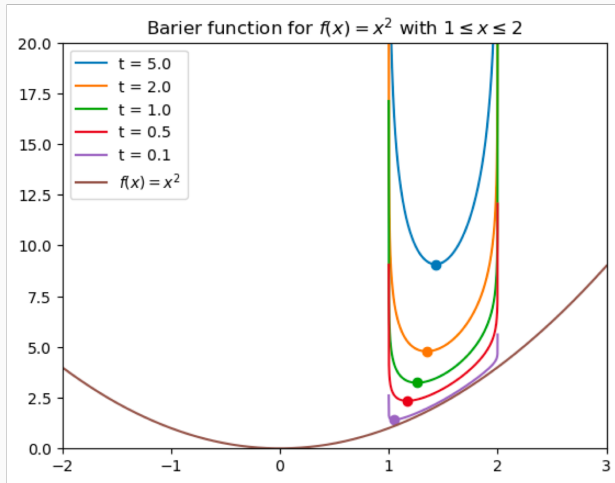


$$\Phi_t(x) = -t \log(2 - x) - t \log(x - 1)$$

On the interval $[1, 2]$:

- $+\infty$ at both endpoints.
- Smooth, strictly convex inside.
- Its unique minimizer is the **analytic center** $x = 1.5$.
- Shrinks uniformly as $t \downarrow 0$, but never lets you touch the boundary.

Barrier + objective: the 1-D central path



$$\min_x x^2 + t [-\log(2-x) - \log(x-1)]$$

What to see:

- $t = 5$: barrier dominates, minimizer near analytic center.
- $t \downarrow 0$: the bowl flattens; minimizer slides toward the unconstrained optimum of x^2 (at $x = 1$, the active boundary).
- Dots trace the **central path** $x^*(t)$ as t sweeps.

Why each $\mu > 0$ is a nice unconstrained problem

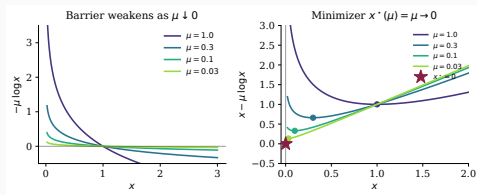
For the toy LP $\min x$ s.t. $x \geq 0$:

$$\min_x x - \mu \log x \Rightarrow x^*(\mu) = \mu.$$

Dual variable: $s(\mu) = \mu/x^*(\mu) = 1$. Complementarity $sx = \mu$ — **not zero, but controlled**.

Two ideas in one slide:

- Smooth *unconstrained* problem per μ .
- Complementarity $s \cdot x = \mu$ — tunable, and $\rightarrow 0$ as $\mu \downarrow 0$.



The barrier problem

For **barrier parameter** $\mu > 0$, define

$$B_\mu(\mathbf{x}) := f(\mathbf{x}) - \mu \sum_{i=1}^m \log(-g_i(\mathbf{x}))$$

and solve the *barrier problem*

$$(P_\mu) \quad \min_{\mathbf{x}} B_\mu(\mathbf{x}) \quad \text{s.t.} \quad h_j(\mathbf{x}) = 0 \quad (j = 1, \dots, p).$$

- Equalities remain; only inequalities are folded into the objective.
- (P_μ) is smooth, with only equality constraints — solvable by Newton's method on the Lagrangian, which you already know.
- Solve (P_μ) for $\mu_1 > \mu_2 > \dots \rightarrow 0$, warm-starting each from the previous solution.

This is called a *sequential unconstrained minimization technique* (SUMT), originated by Fiacco & McCormick (1968). Karmarkar's 1984 LP breakthrough gave it polynomial-time status.

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

Central path: what the KKT of (P_μ) looks like

Stationarity of (P_μ) (no inequalities, just equality constraints):

$$\nabla f(\mathbf{x}) + \mu \nabla \Phi(\mathbf{x}) + \sum_j y_j \nabla h_j(\mathbf{x}) = 0.$$

Expand the barrier gradient:

$$\nabla f(\mathbf{x}) + \sum_i \underbrace{\frac{\mu}{-g_i(\mathbf{x})}}_{=: s_i(\mu)} \nabla g_i(\mathbf{x}) + \sum_j y_j \nabla h_j(\mathbf{x}) = 0.$$

Three observations:

- $s_i(\mu) > 0$ (since $g_i < 0$ at interior points).
- The stationarity equation *matches the KKT stationarity condition exactly* — with s playing the role of the inequality multiplier.
- Complementarity becomes $s_i(\mu) \cdot (-g_i(\mathbf{x})) = \mu$, i.e. **perturbed complementarity**

$s_i \cdot (-g_i(\mathbf{x})) = \mu$

The perturbed KKT system

The **central path** is the curve $\{\mathbf{x}(\mu), \mathbf{y}(\mu), \mathbf{s}(\mu) : \mu > 0\}$ satisfying

$$\nabla f(\mathbf{x}) + \sum_i s_i \nabla g_i(\mathbf{x}) + \sum_j y_j \nabla h_j(\mathbf{x}) = 0 \quad (\text{stationarity})$$

$$h_j(\mathbf{x}) = 0 \quad (\text{eq. feas.})$$

$$g_i(\mathbf{x}) < 0 \quad (\text{ineq. strict feas.})$$

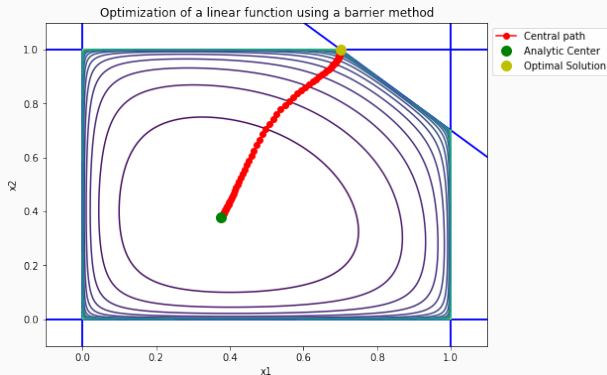
$$s_i > 0 \quad (\text{dual strict feas.})$$

$$s_i \cdot (-g_i(\mathbf{x})) = \mu \quad (\text{perturbed complementarity})$$

Compare with standard KKT: the only change is $s_i g_i = 0$ becomes $s_i \cdot (-g_i) = \mu$. **As $\mu \rightarrow 0$, the central path $\rightarrow$ a KKT point.**

The central path exists and is unique (under standard convexity + Slater) — we'll take this as given. See Nocedal–Wright Thm. 19.2, Boyd–Vandenberghe §11.2.

2D picture: the central path for an LP



$$\min c^\top x \quad \text{s.t.} \quad Ax \leq b$$

The contours are level sets of the log-barrier Φ . As μ sweeps $\infty \rightarrow 0$:

- Large μ : **analytic center** (green).
- Small μ : **optimal vertex** (yellow).
- In between: smooth curve in the strict interior.

Key fact. Each $x^*(\mu)$ is a *unique* minimizer — one well-conditioned Newton problem, no active-set combinatorics.

Worked 2D example: computing the central path explicitly

Problem. $\min -x_1$ s.t. $x_1 + x_2 \leq 1$, $x_1, x_2 \geq 0$. Optimum $\mathbf{x}^* = (1, 0)$, $f^* = -1$.

Barrier problem (slack $s = 1 - x_1 - x_2$):

$$(P_\mu) \min_{x_1, x_2, s > 0} -x_1 - \mu [\log s + \log x_1 + \log x_2].$$

Stationarity ($\partial s / \partial x_i = -1$):

$$\partial_{x_1} : -1 + \mu\left(\frac{1}{s} - \frac{1}{x_1}\right) = 0, \quad \partial_{x_2} : \mu\left(\frac{1}{s} - \frac{1}{x_2}\right) = 0 \Rightarrow s = x_2.$$

From $s = x_2$: $1 - x_1 - x_2 = x_2 \Rightarrow x_2(\mu) = s(\mu) = \frac{1-x_1}{2}$.

Substitute into first equation: $\mu(3x_1 - 1) = x_1(1 - x_1)$, so

$$x_1^2 + (3\mu - 1)x_1 - \mu = 0 \quad \Rightarrow \quad \boxed{x_1(\mu) = \frac{1-3\mu+\sqrt{9\mu^2-2\mu+1}}{2}, \quad x_2(\mu) = \frac{1-x_1}{2}.$$

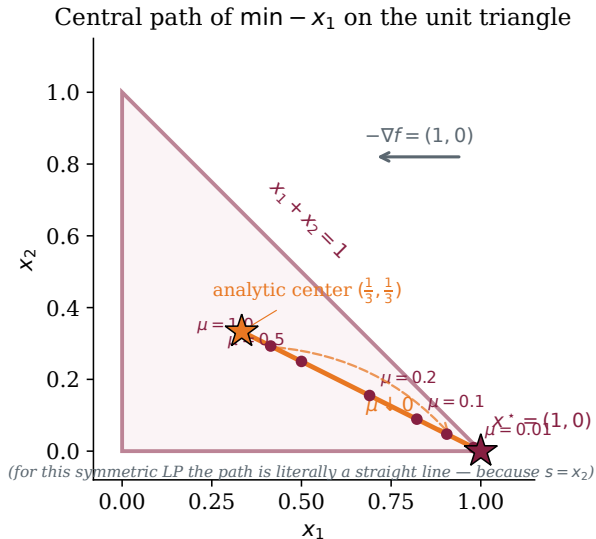
Take $\mu \rightarrow 0$.

μ	1	10^{-1}	10^{-2}	10^{-3}	$\rightarrow 0$
x_1	0.414	0.822	0.980	0.998	$\rightarrow 1$
x_2	0.293	0.089	0.0099	0.00100	$\rightarrow 0$
f	-0.414	-0.822	-0.980	-0.998	$\rightarrow -1$

At $\mu = 0$: $\sqrt{1} = 1 \Rightarrow x_1(0) = 1$, $x_2(0) = 0$. **LP optimum recovered exactly.**✓

For small μ : $x_1 \approx 1 - \mu$, $x_2 \approx \mu/2$. Duality gap $\approx m\mu$ where $m = 3$ is the number of inequality constraints (two bounds + one linear).

Picture: the 2D central path as $\mu \rightarrow 0$



Reading the figure.

- **Triangle:** feasible region $\{x_1 + x_2 \leq 1, x_1, x_2 \geq 0\}$.
- **Orange curve:** central path $\{(x_1(\mu), x_2(\mu))\}$.
- Large μ : path starts at the **analytic center** $(\frac{1}{3}, \frac{1}{3})$.
- $\mu \downarrow 0$: path runs to the **optimal vertex** $(1, 0)$, following $-\nabla f = (1, 0)$.

Why a line, not a curve? For this specific LP the symmetry $s = x_2$ forces $x_2 = (1 - x_1)/2$ for every μ . Most LPs have genuinely curved central paths.

Every LP IPM does exactly this. Choose a warm μ_0 , solve the Newton system to stay near the path, decrease μ , repeat. No active-set combinatorics.

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

Interlude: equality-constrained Newton's method

Before tackling the barrier subproblem, let's review the building block: **Newton's method for equality-constrained minimization**.

Problem.

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{h}(\mathbf{x}) = 0, \quad \mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^p.$$

Lagrangian and KKT.

$$\mathcal{L}(\mathbf{x}, \mathbf{y}) = f(\mathbf{x}) + \mathbf{y}^\top \mathbf{h}(\mathbf{x}).$$

First-order optimality (assuming LICQ) requires

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \mathbf{y}) = \nabla f(\mathbf{x}) + J_{\mathbf{h}}(\mathbf{x})^\top \mathbf{y} = \mathbf{0}, \quad \mathbf{h}(\mathbf{x}) = \mathbf{0}.$$

That's $n + p$ nonlinear equations in $n + p$ unknowns $(\mathbf{x}, \mathbf{y})$.

Idea. Treat these $n + p$ equations as a root-finding problem and apply Newton's method to the *whole pair* $(\mathbf{x}, \mathbf{y})$ simultaneously — primal variables and multipliers.

This is not the only approach. Alternatives include: reduce to the tangent space of $\mathbf{h} = 0$ and run Newton on the reduced problem (null-space methods); penalty methods; augmented Lagrangian. Primal-dual Newton is the most direct and is what IPOPT and SQP solvers use.

Equality-constrained Newton: deriving the KKT system

Stack KKT into a residual vector $F(\mathbf{x}, \mathbf{y})$ in the unknown $\mathbf{z} = (\mathbf{x}, \mathbf{y})$:

$$F(\mathbf{z}) = \begin{pmatrix} \nabla f(\mathbf{x}) + J_h(\mathbf{x})^\top \mathbf{y} \\ \mathbf{h}(\mathbf{x}) \end{pmatrix} = \mathbf{0}.$$

Newton on this system. To solve $F(\mathbf{z}) = 0$, take Newton steps $\mathbf{z}^{k+1} = \mathbf{z}^k + \Delta \mathbf{z}$ where

$$\nabla F(\mathbf{z}^k) \Delta \mathbf{z} = -F(\mathbf{z}^k).$$

Compute the Jacobian ∇F . Each row of F contributes one row-block:

$$\partial_{\mathbf{x}} [\nabla f + J_h^\top \mathbf{y}] = \nabla^2 f(\mathbf{x}) + \sum_{j=1}^p y_j \nabla^2 h_j(\mathbf{x}) =: W, \quad \partial_{\mathbf{y}} [\nabla f + J_h^\top \mathbf{y}] = J_h^\top.$$

$$\partial_{\mathbf{x}} [\mathbf{h}] = J_h, \quad \partial_{\mathbf{y}} [\mathbf{h}] = \mathbf{0}.$$

So:

$$\nabla F = \begin{pmatrix} W & J_h^\top \\ J_h & \mathbf{0} \end{pmatrix}.$$

The Newton KKT system.

$$\begin{pmatrix} W(\mathbf{x}^k, \mathbf{y}^k) & J_h(\mathbf{x}^k)^\top \\ J_h(\mathbf{x}^k) & \mathbf{0} \end{pmatrix} \begin{pmatrix} \Delta \mathbf{x} \\ \Delta \mathbf{y} \end{pmatrix} = - \begin{pmatrix} \nabla f(\mathbf{x}^k) + J_h(\mathbf{x}^k)^\top \mathbf{y}^k \\ \mathbf{h}(\mathbf{x}^k) \end{pmatrix}.$$

Equivalent “full-step” form. Let $\mathbf{y}^+ := \mathbf{y}^k + \Delta \mathbf{y}$. Then (expand, cancel the $J_h^\top \mathbf{y}^k$ term on both sides):

$$\begin{pmatrix} W & J_h^\top \\ J_h & \mathbf{0} \end{pmatrix} \begin{pmatrix} \Delta \mathbf{x} \\ \mathbf{y}^+ \end{pmatrix} = \begin{pmatrix} -\nabla f(\mathbf{x}^k) \\ -\mathbf{h}(\mathbf{x}^k) \end{pmatrix}.$$

Both give the same $\Delta \mathbf{x}$; the second returns the *new* multiplier directly. (Nocedal–Wright uses this form.)

The saddle-point matrix and what it means

The matrix

$$K = \begin{pmatrix} W & J_h^\top \\ J_h & 0 \end{pmatrix} \in \mathbb{R}^{(n+p) \times (n+p)}$$

is called the **KKT matrix** or **saddle-point matrix**.

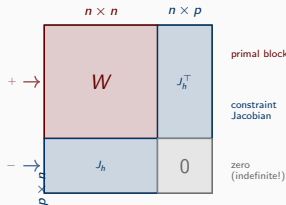
Key properties.

- **Symmetric** (since W is symmetric).
- **Indefinite**: both positive and negative eigenvalues (because of the 0 block). *Not* positive definite; Cholesky doesn't apply.
- **Inertia at solution**: under 2nd-order sufficient conditions + LICQ, K has exactly n_+ , p_- , 0_0 eigenvalues: $\text{inertia}(K) = (n, p, 0)$.
- **Nonsingular** iff J_h full row rank *and* W pos. def. on $\ker J_h$.

How it's solved in practice.

- Sparse symmetric indefinite $LDL^\top$ (Bunch–Kaufman pivoting).
- Industrial: **MA27**, **MA57** (HSL), **MUMPS**, **Pardiso**.
- One factorization + one triangular solve per Newton iteration.

Preview. For (P_μ) , plug the *barrier Hessian and gradient* into this same structure. IPOPT extends to a 3×3 block by treating s as separate variables (primal-dual upgrade, next).



2×2 block form. The 0 block kills positive-definiteness.

Primal barrier method: the naïve algorithm

Path-following template:

1. Pick $\mu_0 > 0$ and a strictly interior $\mathbf{x}^{(0)}$.
2. For $k = 0, 1, 2, \dots$:
 - (a) Solve (approximately) (P_{μ_k}) by Newton's method, starting from $\mathbf{x}^{(k)}$.
 - (b) Let $\mathbf{x}^{(k+1)}$ be the result.
 - (c) Decrease μ : $\mu_{k+1} = \sigma \mu_k$ for some $\sigma \in (0, 1)$ (e.g. $\sigma = 0.1$).
3. Stop when $\mu_k < \epsilon$.

What Newton does on (P_μ) : it's ordinary equality-constrained Newton (which you know) with the Lagrangian

$$\mathcal{L}_\mu(\mathbf{x}, \mathbf{y}) = f(\mathbf{x}) - \mu \sum_i \log(-g_i(\mathbf{x})) + \mathbf{y}^\top \mathbf{h}(\mathbf{x}),$$

and KKT matrix built from $\nabla_x^2 \mathcal{L}_\mu$ and $\nabla \mathbf{h}$.

Problem: the Hessian $\nabla^2 \Phi$ has entries scaling like $1/g_i^2$. Near the boundary these blow up $\Rightarrow$ ill-conditioning. Newton works but needs careful step-control.

Primal-dual interior point (what IPOPT does)

Instead of solving the barrier problem to convergence at each μ_k , **linearize the entire perturbed KKT system** and take *one* Newton step per outer iteration, then decrease μ .

Write the perturbed KKT system (equality constraints only, for clarity):

$$F_\mu(\mathbf{x}, \mathbf{y}, \mathbf{s}) = \begin{pmatrix} \nabla f(\mathbf{x}) + J_g(\mathbf{x})^\top \mathbf{s} + J_h(\mathbf{x})^\top \mathbf{y} \\ \mathbf{h}(\mathbf{x}) \\ S \Sigma(\mathbf{x}) \mathbf{1} - \mu \mathbf{1} \end{pmatrix} = 0,$$

where $S = \text{diag}(s_1, \dots, s_m)$ and $\Sigma(\mathbf{x}) = \text{diag}(-g_1(\mathbf{x}), \dots, -g_m(\mathbf{x}))$. The third block is the vector $(s_i \cdot (-g_i) - \mu)_i$ (perturbed complementarity).

Newton step: solve

$$\nabla F_\mu(\mathbf{x}, \mathbf{y}, \mathbf{s}) \begin{pmatrix} \Delta \mathbf{x} \\ \Delta \mathbf{y} \\ \Delta \mathbf{s} \end{pmatrix} = -F_\mu(\mathbf{x}, \mathbf{y}, \mathbf{s}).$$

Key payoff. $(\mathbf{x}, \mathbf{s})$ and $(\mathbf{y}, \mathbf{s})$ are updated *simultaneously*. The complementarity equation is linearized like any other — no active-set decision needed.

The primal-dual Newton system

Writing the block structure of ∇F_μ :

$$\begin{pmatrix} W(\mathbf{x}, \mathbf{y}, \mathbf{s}) & J_h^\top & J_g^\top \\ J_h & 0 & 0 \\ -S J_g & 0 & \Sigma \end{pmatrix} \begin{pmatrix} \Delta \mathbf{x} \\ \Delta \mathbf{y} \\ \Delta \mathbf{s} \end{pmatrix} = - \begin{pmatrix} \nabla_{\mathbf{x}} \mathcal{L} \\ \mathbf{h} \\ S \Sigma \mathbf{1} - \mu \mathbf{1} \end{pmatrix}$$

where $W = \nabla^2 f + \sum s_i \nabla^2 g_i + \sum y_j \nabla^2 h_j$ is the Hessian of the (true, not barrier) Lagrangian, and $\Sigma = \text{diag}(-g_i(\mathbf{x}))$.

Standard reduction: eliminate $\Delta \mathbf{s}$ via the third row, get a symmetric indefinite system in $(\Delta \mathbf{x}, \Delta \mathbf{y})$:

$$\begin{pmatrix} W + J_g^\top D J_g & J_h^\top \\ J_h & 0 \end{pmatrix} \begin{pmatrix} \Delta \mathbf{x} \\ \Delta \mathbf{y} \end{pmatrix} = \text{rhs}, \quad D = \Sigma^{-1} S = \text{diag}(s_i / (-g_i)).$$

Factorize once per iteration. IPOPT uses MA27/MA57/MUMPS (sparse symmetric-indefinite solvers) for this step. The cost per iteration is dominated by this factorization.

From Lagrangian to Newton system: the logical chain

Step 1 — **Lagrangian**. Attach multipliers $\mathbf{y}$ (equalities) and $\mathbf{s} \geq 0$ (inequalities):

$$\mathcal{L}(\mathbf{x}, \mathbf{y}, \mathbf{s}) = f(\mathbf{x}) + \mathbf{y}^\top \mathbf{h}(\mathbf{x}) + \mathbf{s}^\top \mathbf{g}(\mathbf{x}).$$

Step 2 — **KKT conditions** from stationarity of $\mathcal{L}$ plus feasibility and complementarity:

$$\underbrace{\nabla_{\mathbf{x}} \mathcal{L} = \nabla f + J_g^\top \mathbf{s} + J_h^\top \mathbf{y} = 0}_{\text{stationarity}}, \quad \underbrace{\mathbf{h}(\mathbf{x}) = 0}_{\text{eq. feas.}}, \quad \underbrace{s_i(-g_i(\mathbf{x})) = 0}_{\text{complementarity}}, \quad \mathbf{s} \geq 0, \quad \mathbf{g}(\mathbf{x}) \leq 0.$$

Step 3 — **Perturb the hard equation**. Replace $s_i(-g_i) = 0$ with $s_i(-g_i) = \mu$. Pack the three equality-type conditions into one nonlinear system in the unknown vector $\mathbf{z} = (\mathbf{x}, \mathbf{y}, \mathbf{s})$:

$$F_\mu(\mathbf{z}) = \begin{pmatrix} \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \mathbf{y}, \mathbf{s}) \\ \mathbf{h}(\mathbf{x}) \\ s_i(-g_i(\mathbf{x})) - \mu \end{pmatrix} = \mathbf{0}.$$

(Inequalities $\mathbf{s} > 0$, $\mathbf{g}(\mathbf{x}) < 0$ are enforced *separately* by the fraction-to-boundary rule.)

Step 4 — **Newton's method for a nonlinear system**. To solve $F_\mu(\mathbf{z}) = 0$, iterate $\mathbf{z}^{k+1} = \mathbf{z}^k + \Delta \mathbf{z}$ where

$$\boxed{\nabla F_\mu(\mathbf{z}^k) \Delta \mathbf{z} = -F_\mu(\mathbf{z}^k)}.$$

This is *the* matrix system. ∇F_μ is the **Jacobian** of the residual vector — a matrix whose (i, j) entry is $\partial F_i / \partial z_j$.

Step 5 — **Why “block” structure?** F_μ is naturally partitioned into three row-blocks (stationarity, equality, complementarity) and $\mathbf{z}$ into three column-blocks $(\mathbf{x}, \mathbf{y}, \mathbf{s})$. So ∇F_μ is a 3×3 grid of **blocks**:

$$\nabla F_\mu = \begin{pmatrix} \partial(\nabla_{\mathbf{x}} \mathcal{L}) / \partial \mathbf{x} & \partial(\nabla_{\mathbf{x}} \mathcal{L}) / \partial \mathbf{y} & \partial(\nabla_{\mathbf{x}} \mathcal{L}) / \partial \mathbf{s} \\ \partial \mathbf{h} / \partial \mathbf{x} & \partial \mathbf{h} / \partial \mathbf{y} & \partial \mathbf{h} / \partial \mathbf{s} \\ \partial[S(-\mathbf{g}) - \mu] / \partial \mathbf{x} & \partial[\cdot] / \partial \mathbf{y} & \partial[\cdot] / \partial \mathbf{s} \end{pmatrix}.$$

The next slide fills in these nine entries. Most are zero or immediate; only the Hessian-of-Lagrangian block W requires work.

Deriving the Newton step block by block

Let J_g (rows $\nabla g_i^\top$), J_h be Jacobians; $S = \text{diag}(s_i)$, $\Sigma(\mathbf{x}) = \text{diag}(-g_i(\mathbf{x}))$ ($\succ 0$ interior). Perturbed-KKT residuals:

$$F_\mu = \begin{pmatrix} r_1 \\ r_2 \\ r_3 \end{pmatrix} = \begin{pmatrix} \nabla f + J_g^\top \mathbf{s} + J_h^\top \mathbf{y} \\ \mathbf{h}(\mathbf{x}) \\ S\Sigma\mathbf{1} - \mu\mathbf{1} \end{pmatrix}, \quad [S\Sigma\mathbf{1}]_i = s_i(-g_i).$$

Differentiate each block. $\partial_{\mathbf{x}} r_1 = \nabla^2 f + \sum s_i \nabla^2 g_i + \sum y_j \nabla^2 h_j =: W$; $\partial_{\mathbf{y}} r_1 = J_h^\top$, $\partial_{\mathbf{s}} r_1 = J_g^\top$. $\partial_{\mathbf{x}} r_2 = J_h$. $\partial_{\mathbf{x}} r_3 = -SJ_g$, $\partial_{\mathbf{s}} r_3 = \Sigma$.

$$\nabla F_\mu = \begin{pmatrix} W & J_h^\top & J_g^\top \\ J_h & 0 & 0 \\ -SJ_g & 0 & \Sigma \end{pmatrix}.$$

Reduce to symmetric indefinite. From row 3 with $\Sigma \succ 0$: $\Delta \mathbf{s} = \underbrace{\Sigma^{-1} S}_{=: D} J_g \Delta \mathbf{x} - \Sigma^{-1} r_3$. Substitute:

$$\begin{pmatrix} W + J_g^\top D J_g & J_h^\top \\ J_h & 0 \end{pmatrix} \begin{pmatrix} \Delta \mathbf{x} \\ \Delta \mathbf{y} \end{pmatrix} = - \begin{pmatrix} r_1 - J_g^\top \Sigma^{-1} r_3 \\ r_2 \end{pmatrix}.$$

IPOPT factorizes this with MA27/MA57/MUMPS. Recover $\Delta \mathbf{s}$ from the reduction.

$D = \text{diag}(s_i/(-g_i))$ is the *barrier-weighted* active-constraint penalty: as $-g_i \rightarrow 0$, $D_{ii} \rightarrow \infty$, enforcing the constraint.

Worked example: one primal-dual Newton iteration

Problem. $\min x^2$ s.t. $x \geq 1$. Rewrite $g(x) = 1 - x \leq 0$. Optimum $(x^*, s^*) = (1, 2)$.

Central path. Barrier stationarity $2x - \mu/(x - 1) = 0$ gives $x(\mu) = \frac{1+\sqrt{1+2\mu}}{2}$, $s(\mu) = 2x(\mu)$. As $\mu \downarrow 0$: $(x, s) \rightarrow (1, 2)$. ✓

Ingredients. $\nabla f = 2x$, $J_g = -1$, $W = 2$, $\Sigma = x - 1$. Residuals: $r_1 = 2x - s$, $r_3 = s(x - 1) - \mu$. Jacobian:

$$\nabla F_\mu = \begin{pmatrix} W & J_g^\top \\ -SJ_g & \Sigma \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ s & x - 1 \end{pmatrix}.$$

Iterate 0. $(x^0, s^0) = (2, 2)$, $\mu_0 = 1$, $r_1 = 2$, $r_3 = 1$. Solve $\begin{pmatrix} 2 & -1 \\ s & x - 1 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \end{pmatrix} = - \begin{pmatrix} 2 \\ 1 \end{pmatrix} \Rightarrow \Delta x = -0.75$, $\Delta s = 0.5$.

Fraction-to-boundary ($\tau = 0.995$): $1 - 0.75\alpha \geq 0.005 \Rightarrow \alpha \leq 1.327$; slack always OK. Full step

$$\alpha = 1 \Rightarrow (x^1, s^1) = (1.25, 2.5).$$

Progress. $r_1 = 0$ exact; $s^1(x^1 - 1) = 0.625$ (was 1). Decrease $\mu_1 = 0.25$; repeat.

Iterate 1 at $(1.25, 2.5)$, $\mu = 0.25$: $r_3 = 0.375$. Solve $\Rightarrow \Delta x = -0.125$, $\Delta s = -0.25$. $(x^2, s^2) = (1.125, 2.25)$.

Central-path at $\mu = 0.25$: $(1.112, 2.225)$. Within 0.025 — converging to $(1, 2)$. ✓

2D worked example: J_g , W , and the 3×3 Jacobian by hand

Problem. $\min f(\mathbf{x}) = x_1^2 + x_2^2$ s.t. $g_1(\mathbf{x}) = 1 - x_1 \leq 0$, $g_2(\mathbf{x}) = 1 - x_2 \leq 0$. Optimum $\mathbf{x}^* = (1, 1)$, $\mathbf{s}^* = (2, 2)$.

Ingredients (dimensions: $n = 2$, $m = 2$, $p = 0$).

$$\nabla f(\mathbf{x}) = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix}, \quad J_g(\mathbf{x}) = \begin{pmatrix} \nabla g_1^\top \\ \nabla g_2^\top \end{pmatrix} = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}, \quad W = \nabla^2 f + \sum s_i \nabla^2 g_i = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix},$$

$$\Sigma(\mathbf{x}) = \text{diag}(-g_i) = \text{diag}(x_1 - 1, x_2 - 1), \quad S = \text{diag}(s_1, s_2).$$

Residuals at current $(\mathbf{x}, \mathbf{s})$:

$$\mathbf{r}_1 = \nabla f + J_g^\top \mathbf{s} = \begin{pmatrix} 2x_1 - s_1 \\ 2x_2 - s_2 \end{pmatrix}, \quad \mathbf{r}_3 = S\Sigma\mathbf{1} - \mu\mathbf{1} = \begin{pmatrix} s_1(x_1 - 1) - \mu \\ s_2(x_2 - 1) - \mu \end{pmatrix}.$$

4×4 Jacobian ($n + m = 4$; $p = 0$, no equalities):

$$\nabla F_\mu = \begin{pmatrix} W & J_g^\top \\ -SJ_g & \Sigma \end{pmatrix} = \begin{pmatrix} 2 & 0 & -1 & 0 \\ 0 & 2 & 0 & -1 \\ s_1 & 0 & x_1 - 1 & 0 \\ 0 & s_2 & 0 & x_2 - 1 \end{pmatrix}.$$

Iterate at $(\mathbf{x}, \mathbf{s}) = (2, 2, 2, 2)$, $\mu = 1$. $\mathbf{r}_1 = (2, 2)$, $\mathbf{r}_3 = (2 \cdot 1 - 1, 2 \cdot 1 - 1) = (1, 1)$. The 4×4 Jacobian is block-diagonal into two identical 2×2 systems (problem separates!):

$$\begin{pmatrix} 2 & -1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} \Delta x_i \\ \Delta s_i \end{pmatrix} = - \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad \forall i \Rightarrow \Delta x_i = -0.75, \Delta s_i = 0.5.$$

Full step $\Rightarrow \mathbf{x}^1 = (1.25, 1.25)$, $\mathbf{s}^1 = (2.5, 2.5)$. Same arithmetic as the 1D example, *twice* — the separability makes it transparent.

Step acceptance: fraction-to-the-boundary rule

After computing $(\Delta \mathbf{x}, \Delta \mathbf{y}, \Delta \mathbf{s})$, we need to step without leaving the strict interior.

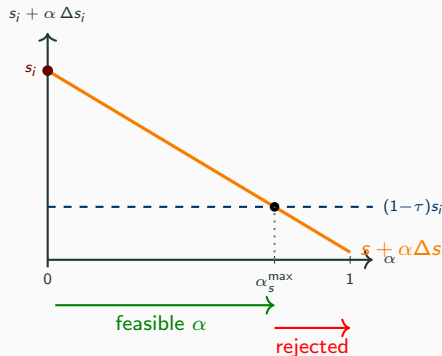
Fraction-to-the-boundary with $\tau_k \in (0, 1)$ (e.g. $\tau_k = 0.995$):

$$\alpha_s^{\max} = \max\{\alpha \in (0, 1] : s_i + \alpha \Delta s_i \geq (1 - \tau_k)s_i \forall i\}.$$

(Max over α , but inequality must hold for *all* i ; equivalent to the per-component $\min_i$.) Similar bound for $\mathbf{x}$ against $g_i(\mathbf{x}) \leq 0$.

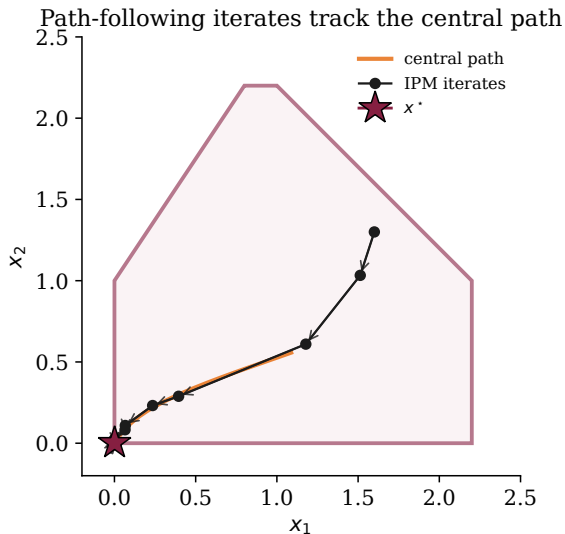
Then line-search on a merit function / filter (next slide) starting from $\alpha^{\max}$ downward.

- Keeps $\mathbf{x}$ in strict interior and $\mathbf{s} > 0$ throughout.
- Stays on or near the central path.
- $\tau_k \rightarrow 1$ as $\mu_k \rightarrow 0$ — allow longer steps near optimality.



The longest α we allow before the slack would drop below $(1 - \tau)s_i$.

Picture: iterates tracking the central path



Primal-dual Newton +
fraction-to-boundary, started from an
off-path interior point.

- Early iterates *swing* toward the central path (centering).
- Once on the path, each step roughly halves μ and slides along the curve.
- Short-step analysis: polynomial complexity $O(\sqrt{n} \log(1/\varepsilon))$ Newton iterations (n = number of complementarity pairs).

Fraction-to-boundary by hand: numerical example

Setup. Current slacks $\mathbf{s} = (1.0, 0.5, 2.0)$. Newton step $\Delta\mathbf{s} = (-0.2, -0.6, +0.4)$. Use $\tau = 0.995$, so floor $(1 - \tau)\mathbf{s} = (0.005, 0.0025, 0.01)$.

Per-component bound. Solve $s_i + \alpha\Delta s_i \geq (1 - \tau)s_i$ for each i :

$$\alpha_i = \begin{cases} \frac{-s_i(1 - (1 - \tau))}{\Delta s_i} = \frac{-\tau s_i}{\Delta s_i} & \text{if } \Delta s_i < 0, \\ +\infty & \text{if } \Delta s_i \geq 0 \end{cases}$$

i	s_i	Δs_i	$(1 - \tau)s_i$	sign	α_i
1	1.0	-0.2	0.005	< 0	$-0.995(1.0)/(-0.2) = \mathbf{4.975}$
2	0.5	-0.6	0.0025	< 0	$-0.995(0.5)/(-0.6) = \mathbf{0.829}$
3	2.0	+0.4	0.01	≥ 0	$+\infty$

Combine. $\alpha_s^{\max} = \min(1, \min_i \alpha_i) = \min(1, 0.829) = \mathbf{0.829}$. Component $i=2$ is *binding*: s_2 would hit its floor at $\alpha = 0.829$. Take the step, and verify:

$$\mathbf{s} + 0.829 \Delta\mathbf{s} = (0.834, 0.003, 2.332), \quad s_2 = 0.003 \geq 0.0025 \checkmark$$

The $\min_i$ is essential: one tight component throttles the whole step. A good starting point has *balanced* slacks so no component dominates $\alpha^{\max}$.

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

The classic Hock–Schittkowski problem (HS071)

$$\begin{aligned} \min_{x_1, \dots, x_4} \quad & x_1 x_4 (x_1 + x_2 + x_3) + x_3, \\ \text{s.t.} \quad & x_1 x_2 x_3 x_4 \geq 25, \quad x_1^2 + x_2^2 + x_3^2 + x_4^2 = 40, \\ & 1 \leq x_i \leq 5, \quad i = 1, \dots, 4. \end{aligned}$$

Optimum: $\mathbf{x}^* \approx (1.00, 4.74, 3.82, 1.38)$, $f^* \approx 17.01$.

```
import pyomo.environ as pyo
m = pyo.ConcreteModel()
m.x = pyo.Var([1,2,3,4], bounds=(1, 5), initialize=1.0)
m.obj = pyo.Objective(
    expr = m.x[1]*m.x[4]*(m.x[1]+m.x[2]+m.x[3]) + m.x[3])
m.c1 = pyo.Constraint(expr = m.x[1]*m.x[2]*m.x[3]*m.x[4] >= 25)
m.c2 = pyo.Constraint(expr = sum(m.x[i]**2 for i in [1,2,3,4]) == 40)

pyo.SolverFactory("ipopt").solve(m, tee=True)
```

Reading the IPOPT iteration log

```
iter   objective   inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr ls
  0  1.6109693e+01  1.12e+01  5.28e-01 -1.0  0.00e+00 -  0.00e+00  0.00e+00  0
  1  1.6384032e+01  8.37e+00  1.42e+01 -1.0  2.12e+00 -  7.86e-01  2.49e-01  2f
  2  1.7008856e+01  8.37e-03  1.71e+00 -1.0  1.57e+00 -  1.00e+00  1.00e+00h 1
  3  1.6993368e+01  7.50e-04  1.30e-01 -2.5  6.83e-02 -  1.00e+00  9.99e-01h 1
  4  1.7014114e+01  1.15e-05  8.23e-04 -3.8  1.05e-02 -  1.00e+00  1.00e+00h 1
  5  1.7014017e+01  7.06e-09  7.23e-07 -5.7  1.34e-04 -  1.00e+00  1.00e+00h 1
  6  1.7014017e+01  3.05e-14  2.79e-11 -8.6  6.50e-08 -  1.00e+00  1.00e+00h 1
```

One row per outer Newton iteration. Columns decoded:

iter iteration number (iter 0 = starting point, before any step)
objective current value of $f(\mathbf{x}^k)$ — *not* the barrier objective
inf_pr primal infeasibility $\max(\|\mathbf{h}(\mathbf{x})\|_\infty, \max_i \max(g_i(\mathbf{x}), 0))$
inf_du dual infeasibility $\|\nabla_x \mathcal{L}\|_\infty$ (the r_1 residual)
lg(mu) $\log_{10} \mu$ — barrier parameter. $-1.0 \rightarrow -8.6$ here: six μ updates
||d|| ∞ -norm of the Newton search direction $(\Delta \mathbf{x}, \Delta \mathbf{y}, \Delta \mathbf{s})$
lg(rg) $\log_{10}$ of inertia regularization δ added to W ; - when $\delta = 0$
alpha_du dual step length (applied to $\mathbf{s}$ update, capped by fraction-to-boundary)
alpha_pr primal step length (applied to $\mathbf{x}$, after filter line search)
ls # of line-search trials; *letter* = step type (see below)

Line-search letter codes (after alpha_pr): f = f-type (objective-decreasing) step, filter *not* augmented; h = h-type (feasibility-decreasing), filter *augmented*; R = restoration phase entered; F/H = f/h-type while in restoration; s = accepted by “soft restoration”; r = second-order correction used.

Everything visible here comes from the theory we built. The μ decrease $-1.0 \rightarrow -2.5 \rightarrow -3.8 \rightarrow -5.7 \rightarrow -8.6$ is the outer loop, and each row is *one* linearized-KKT Newton step. inf_pr and inf_du are the two residual norms you minimize; once both are below tolerance and μ is small, IPOPT declares optimality.

What IPOPT prints at the end: final summary

After the iteration table, IPOPT prints a diagnostic block and an **exit status**:

```
Number of Iterations.....: 6
                             (scaled)   (unscaled)
Objective.....: 1.7014017e+01  1.7014017e+01
Dual infeasibility.....: 2.79e-11    2.79e-11
Constraint violation...: 3.05e-14    3.05e-14
Complementarity.....: 2.51e-09    2.51e-09
Overall NLP error.....: 2.51e-09    2.51e-09
```

```
obj/grad/Jac evals.....: 7 / 7 / 7
Lagrangian Hessian evals: 6
Total CPU secs in IPOPT.: 0.012
```

EXIT: Optimal Solution Found.

Decoding each row:

- **Dual infeas.:** $\|\nabla_x \mathcal{L}\|_\infty$ — “is this a KKT point?”
- **Constraint viol.:** $\max(\|\mathbf{h}\|_\infty, \max_i g_i(\mathbf{x})_+)$ — feasibility.
- **Complementarity:** $\|\mathbf{S}\mathbf{\Sigma}\mathbf{1}\|_\infty$ — inactive multipliers zero?
- **Overall NLP error:** scaled max of the three above; compared to `tol` (default 10^{-8}) to declare convergence.
- **Scaled vs unscaled:** scaled uses IPOPT’s internal scaling; report unscaled values in papers.

IPOPT exit messages: not every “exit” is success

The most common EXIT: messages:

Optimal Solution Found	Scaled NLP error $< \text{tol}$. The only true success.
Solved To Acceptable Level	Below looser <code>acceptable_tol</code> . <i>Verify residuals manually.</i>
Converged to a point of local infeasibility	Stationary point of $\ h\ ^2$ found but not feasible.
Maximum Number of Iterations Exceeded	Hit <code>max_iter</code> (default 3000).
Restoration Phase Failed	Usually a modeling error or bad scaling.
Invalid Number in NLP Function	NaN/Inf in f, g, h or derivatives — callback bug.

The trap. Students routinely read “Solved To Acceptable Level” as success. It’s not. It means `tol` was *not* reached, but `acceptable_tol` (10^{-6} by default) was, in several consecutive iterations. For reported results, always check Dual infeasibility and Constraint violation manually.

What Pyomo gives back after solve()

The iteration log and final summary are IPOPT's *stdout*. Pyomo separately returns a results object and writes the solution onto the model:

```
results = pyo.SolverFactory("ipopt").solve(m, tee=True)

# 1. Termination status (cross-solver, normalized by Pyomo)
print(results.solver.status)           # 'ok'
print(results.solver.termination_condition) # 'optimal'

# 2. Solution values --- written onto m.x, m.obj, etc.
for i in [1,2,3,4]:
    print(f"x[{i}] = {pyo.value(m.x[i]):.4f}")
print(f"f(x*) = {pyo.value(m.obj):.4f}")

# 3. Dual multipliers (require suffix declaration BEFORE solving)
m.dual = pyo.Suffix(direction=pyo.Suffix.IMPORT)
# ... solve ...
print(m.dual[m.c1]) # multiplier on x1*x2*x3*x4 >= 25
print(m.dual[m.c2]) # multiplier on sum x_i^2 == 40
```

`tee=True` streams IPOPT's *stdout* to your terminal; without it the iteration log is silent but the results object is identical.

Where the solution lives after solve()

What lives where on the model:

- `pyo.value(m.x[i])` — the primal $\mathbf{x}^*$.
- `pyo.value(m.obj)` — the objective at the optimum.
- `m.dual[c]` — Lagrange multiplier for constraint c (y_j for equality, s_i for inequality).
*Requires `m.dual = pyo.Suffix(direction=pyo.Suffix.IMPORT)` **before** solving.*
- `m.rc[v]` — reduced cost / bound multiplier for variable v .
- `results.solver.termination_condition` — optimal, infeasible, maxIterations, error, Always branch on this *before* using the solution.

Sign convention. Pyomo+IPOPT returns duals with the sign of $\mathcal{L} = f + \sum y_j h_j + \sum s_i g_i$. For a $\leq$ constraint, a *negative* multiplier means “tightening the constraint would *worsen* the objective.” Different solvers use different conventions — sanity-check on a small example.

What to watch for when reading IPOPT logs

Healthy log

`inf_pr` and `inf_du` decrease smoothly; `lg(mu)` decreases smoothly; `alpha_pr` = 1.0 most iterations; `ls` small.

Unhealthy signs

- `inf_pr` plateaus $\Rightarrow$ infeasible or bad scaling.
- Many small `alpha_pr` $\Rightarrow$ line search rejecting; search direction bad.
- `ls` sustained $> 4 \Rightarrow$ filter thrashing.
- `lg(rg)` appears $\Rightarrow$ inertia correction active (nonconvexity).
- “Solved to acceptable level” $\neq$ “Solved”. Verify residuals yourself.

We'll come back to the theory behind these quantities — why the outer loop converges, why Newton converges at each μ , and what Mehrotra's trick buys us.

Aside: Pyomo vs. CasADi — choosing a modeling layer

Both front-end IPOPT (and other solvers). They have different sweet spots — neither is strictly better.

Pyomo's strengths.

- True *algebraic modeling language*: `model.x[i,j]` over index sets, constraints as Python functions over those sets. Reads like mathematical notation.
- First-class **MILP / MINLP**: interfaces to Gurobi, CPLEX, CBC, Bonmin, Couenne, SCIP.
- Structured extensions: `pyomo.dae` (differential-algebraic systems), `pyomo.gdp` (generalized disjunctive programming), `mpi-sppy` / PySP (stochastic programming, scenario decomposition).
- Solver-agnostic — swap solvers with one line.

CasADi's strengths.

- **Symbolic AD** is fast and robust for highly nonlinear problems: exact derivatives, optimized expression graph. Function + derivative evaluations typically much faster than Pyomo's (matters when solvers call them thousands of times).
- Built-in **ODE/DAE integrators with sensitivities** (SUNDIALS), composing naturally with the optimizer.
- **C code generation**: export the NLP + derivatives to standalone C. This is why CasADi dominates embedded MPC.
- The **MX graph** lets you embed integrators, NLP solvers, or root-finders as nodes inside a larger problem (bilevel, multi-stage, sensitivity-based formulations).

Rough rule of thumb.

- MILP / MINLP, large structured LP, stochastic programming, algebraic modeling over index sets → **Pyomo**.
- Continuous NLP (especially nonconvex), optimal control, MPC, parameter estimation, ODEs/DAEs in constraints, anything needing speed or hardware deployment → **CasADi**.
- People often use *both*: CasADi for the continuous nonlinear subproblem, Pyomo (or `gurobipy`) for the integer master in a decomposition scheme.

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

Why Newton? Local quadratic convergence

The inner solver at each μ is Newton's method on the perturbed KKT system $F_\mu = 0$. Why is that the right choice?

Theorem [Nocedal–Wright Thm. 11.2, informally]. If ∇F_μ is Lipschitz near a solution $\mathbf{z}^*(\mu)$ and is nonsingular there, then Newton's method started close enough to $\mathbf{z}^*(\mu)$ satisfies

$$\|\mathbf{z}^{k+1} - \mathbf{z}^*\| \leq C \|\mathbf{z}^k - \mathbf{z}^*\|^2.$$

That is, the *number of correct digits doubles per iteration* once you are close.

Why this matters for IPM:

- At each μ , we don't need to solve exactly — a few Newton steps drops the residual to $O(\mu)$.
- Decreasing μ by a constant factor and warm-starting from the previous $\mathbf{z}^*(\mu)$ keeps us inside the quadratic-convergence region.
- Short-step analysis (next slide) turns this into a provable polynomial bound.

The alternative. First-order methods on the barrier problem converge linearly — you'd need $\log(1/\varepsilon)$ *per* μ , times $\log(1/\varepsilon)$ outer updates. Newton wins because inner cost is constant.

Short-step IPM: polynomial-time complexity

Setting. LP-like case: f linear, only bound constraints $x \geq 0$. Let n = number of such nonneg constraints (= number of complementarity pairs). Here $X = \text{diag}(x_i)$, $S = \text{diag}(s_i)$.

Proximity measure.

$$\delta(x, s; \mu) = \left\| \frac{XS e}{\mu} - e \right\|_2, \quad X = \text{diag}(x_i), \quad S = \text{diag}(s_i).$$

$\delta = 0 \Leftrightarrow$ on the central path. A **neighborhood** $\mathcal{N}_2(\theta) = \{\delta \leq \theta\}$ is a tube around the path.

Short-step path-following algorithm.

1. Start with $(x^0, s^0) \in \mathcal{N}_2(\theta)$, $\theta \in (0, 1)$.
2. At iteration k : set $\mu_{k+1} = (1 - \sigma/\sqrt{n})\mu_k$ for a small $\sigma > 0$.
3. Take one full Newton step on the perturbed KKT at μ_{k+1} .
4. Analysis shows the new iterate remains in $\mathcal{N}_2(\theta)$.

Theorem [Nocedal–Wright Thm. 14.4; Renegar 1988]. After

$$K = O\left(\sqrt{n} \log \frac{n\mu_0}{\varepsilon}\right)$$

iterations, the duality gap $n\mu_K \leq \varepsilon$.

Intuition. Each iteration:

- Shrinks μ by a factor $1 - \sigma/\sqrt{n}$ (slow, because $\sqrt{n}$ in denominator).
- Needs only *one* Newton step (because of quadratic convergence within the neighborhood).

Product: total Newton steps scale like $\sqrt{n} \log(1/\varepsilon)$ — **polynomial in n and $\log(1/\varepsilon)$.**

Historical context. This is the complexity result that broke the simplex-is-exponential-worst-case barrier for LP. Karmarkar (1984) — projective-scaling IPM, $O(n^{3.5}L)$. Renegar (1988) — path-following IPM, $O(\sqrt{n}L)$. Nesterov–Nemirovski (1994) — self-concordant barriers, general convex cones.

How the polynomial complexity bound is proved (Wright)

Polynomial complexity follows from a per-iteration decrease of the form

$$\mu_{k+1} \leq (1 - Cn^{-\nu}) \mu_k, \quad k = 0, 1, 2, \dots,$$

where $C > 0$ is independent of n and $\nu \geq \frac{1}{2}$ for conventional methods.

Such rates require delicate manipulation of the **modified Newton** strategy for computing steps and of the **choice of step length** α_k .

Standard telescoping argument. To drive $\mu_K \leq \varepsilon \mu_0$:

$$\begin{aligned} \mu_K \leq \varepsilon \mu_0 &\Leftrightarrow (1 - Cn^{-\nu})^K \leq \varepsilon \\ &\Leftrightarrow K \log(1 - Cn^{-\nu}) \leq \log \varepsilon \\ &\Leftrightarrow K (-Cn^{-\nu}) \leq \log \varepsilon \quad \text{since } \log(1 + t) \leq t \\ &\Leftrightarrow K \geq n^\nu |\log \varepsilon| / C. \end{aligned}$$

Conclusion. We get $\mu_K / \mu_0 \leq \varepsilon$ in

$$K = O(n^\nu |\log \varepsilon|) \text{ iterations.}$$

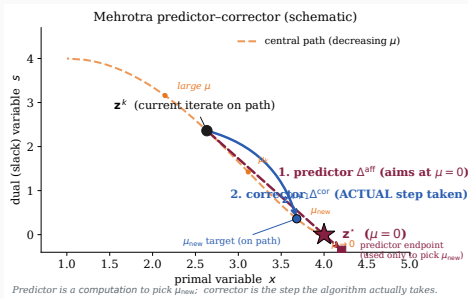
Why $\nu = \frac{1}{2}$ is special. Short-step analysis gives $\nu = \frac{1}{2}$, yielding the celebrated $O(\sqrt{n} \log(1/\varepsilon))$ bound. Long-step / predictor-corrector methods achieve $\nu = 1$ worst-case but are vastly faster in practice. The key ingredient is always: (i) a rate $(1 - Cn^{-\nu})$ per iteration, (ii) the inequality $\log(1 + t) \leq t$ to linearize, (iii) telescope over K iterations.

What IPOPT actually does: Mehrotra's predictor-corrector

Short-step is slow. Industrial solvers use **long steps** with *adaptive* μ .

Mehrotra 1992 — three steps per iteration:

1. **Predictor:** solve Newton with $\mu = 0$ from $\mathbf{z}^k$. *Don't take this step.* Just compute the resulting complementarity μ_{aff} .
2. **Adaptive μ :** pick $\mu_{\text{new}} = \sigma \mu_k$ with $\sigma = (\mu_{\text{aff}} / \mu_k)^3$. (Predictor was aggressive $\Rightarrow \sigma$ small $\Rightarrow$ bold μ decrease.)
3. **Corrector:** resolve Newton with $\mu_{\text{new}} +$ a 2nd-order correction. *This is the step taken.*



IPOPT's variant.

- Monotone:
$$\mu_{k+1} = \max(\varepsilon/10, \min(\kappa_\mu \mu_k, \mu_k^{\theta_\mu})).$$
- Adaptive: Mehrotra-style, driven by complementarity.

Practical behavior. On well-behaved NLPs, IPOPT converges in 20–100 Newton iterations regardless of problem size (HS071 takes 6). Each iteration is dominated by the sparse symmetric-indefinite solve.

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

Do we need a Phase I? Primal-dual initialization

Recall simplex. Classical simplex requires a *basic feasible solution* to start. If you don't have one, you run **Phase I**: a separate LP that constructs a feasible vertex (e.g. Big-M or the two-phase method).

Modern primal-dual IPM — no Phase I needed. You only need

$$\mathbf{s}^0 > 0 \quad \text{and} \quad -g_i(\mathbf{x}^0) > 0 \quad \forall i \quad (\text{strict “interior” of bounds/slacks})$$

The *equality* constraints $\mathbf{h}(\mathbf{x}) = 0$ need **not** hold initially — the Newton system drives all three residuals r_1, r_2, r_3 to zero *simultaneously*. This is the **infeasible-start primal-dual IPM** (Kojima–Megiddo–Mizuno 1993; Wright 1997, Ch. 6).

Three ways different solvers handle “Phase I”:

- **Infeasible-start:** Start anywhere with $\mathbf{s}, -\mathbf{g} > 0$. Let Newton handle it. *IPOPT*, *KNITRO*, *LOQO*.
- **Homogeneous self-dual (HSD) embedding:** Embed the LP/conic program in a bigger problem with a trivial strictly feasible starting point; the solution reveals optimal, primal-infeasible, or dual-infeasible. *MOSEK*, *ECOS*, *SeDuMi*, *SCS*.
- **Big-M / penalty:** Classical. Rarely used in modern IPM implementations.

Moral. For IPOPT, you don't construct a feasible point *up front* — you just need the *strict interior* of the bounds and inequality constraints. Equality feasibility is part of what Newton solves.

How IPOPT actually initializes the iterates

Starting from your $\mathbf{x}^0$ (and optionally $\mathbf{y}^0, \mathbf{s}^0$), IPOPT performs four steps:

- 1. Slack reformulation.** Every inequality $g_i(\mathbf{x}) \leq 0$ is internally rewritten as $g_i(\mathbf{x}) + s_i = 0, s_i \geq 0$. IPOPT's internal problem has only equalities + bound constraints.
- 2. Push-to-interior for bounds.** Any x_i^0 at or outside $[x_L, x_U]$ is shifted strictly inside by a small fraction (options `bound_push`, `bound_frac`, defaults 10^{-2}). Slacks start at $s_i^0 = \max(1, |g_i(\mathbf{x}^0)|)$.
- 3. Least-squares multiplier estimate.** With $\mathbf{x}^0, \mathbf{s}^0$ fixed, IPOPT solves for $\mathbf{y}^0$ to minimize the initial dual residual $\|\nabla f + J_g^\top \mathbf{s}^0 + J_h^\top \mathbf{y}\|_2$ (option `least_square_init_duals` yes). Falls back to $\mathbf{y}^0 = 0$ if ill-conditioned.
- 4. Restoration phase (on-demand Phase I).** If the filter line search cannot accept any step, IPOPT temporarily minimizes a feasibility measure $\|\mathbf{h}(\mathbf{x})\|_1 + \|\mathbf{g}(\mathbf{x}) + \mathbf{s}^e\|_1 + \frac{\zeta}{2} \|D(\mathbf{x} - \mathbf{x}_R)\|_2^2$ (here $\mathbf{s}^e$ are the slack variables on inequalities — not the dual multipliers) to pull the iterate back toward feasibility, then resumes. *This* is IPOPT's Phase I — but it fires on demand, not at the start.

A good starting point saves iterations. In Pyomo: `pyo.Var(initialize=value)` or `m.x[i].set_value(v)`.
Supply warm-start multipliers when re-solving perturbed problems (option `warm_start_init_point` yes).

Why a globalization step is needed

The Newton step $\Delta \mathbf{z}$ is a *local* object — it's the right answer in a neighborhood of the central-path point, but not necessarily far away.

Failure modes without globalization:

- Full Newton step increases the objective dramatically.
- Full Newton step leaves the strict interior (partially handled by fraction-to-boundary, but still possible for equality violation).
- Full Newton step on a nonconvex problem lands at a non-descent direction.
- Full step oscillates between points of equal “merit” (cycling).

Classical solution: a merit function.

$$\phi_{\pi}(\mathbf{x}) = f(\mathbf{x}) + \pi \|\mathbf{h}(\mathbf{x})\|_1, \quad \pi > 0$$

and do backtracking line search on ϕ_{π} .

Problem: choosing π .

- π too small: solver tolerates infeasibility, drifts away from $\{h = 0\}$.
- π too large: rejects steps that temporarily worsen $\|h\|$, stalling progress.
- Good choice depends on current iterate *and* step — no universal rule.

The Maratos effect: when merit functions kill Newton

Maratos (1978). There exist equality-constrained problems where the *full* Newton step (with superlinear convergence) is rejected by every ℓ_1 merit function ϕ_π for every π .

Toy example. Minimize $f(x_1, x_2) = 2(x_1^2 + x_2^2 - 1) - x_1$ subject to $h(x_1, x_2) = x_1^2 + x_2^2 - 1 = 0$ (the unit circle). Optimum at $(1, 0)$.

From a point on the circle near $(1, 0)$, the Newton step is tangential and overshoots the circle slightly. Then:

- $\|h\|$ *increases* (step leaves circle).
- f decreases only to second order.
- $\phi_\pi = f + \pi\|h\|$ *increases* for every $\pi > 0 \Rightarrow$ step rejected.

Cost of rejection. Line search shrinks α , loses quadratic convergence, solver slows to linear rate.

Classical fixes.

- Second-order correction (Fletcher): add a correction term after the Newton step.
- Watchdog technique: accept non-monotone iterates under a “relaxation” rule.
- **Filter methods** (Fletcher–Leyffer 2002): treat f and $\|h\|$ as separate objectives. No penalty parameter.

Wächter–Biegler filter: the big idea

Two objectives, not one.

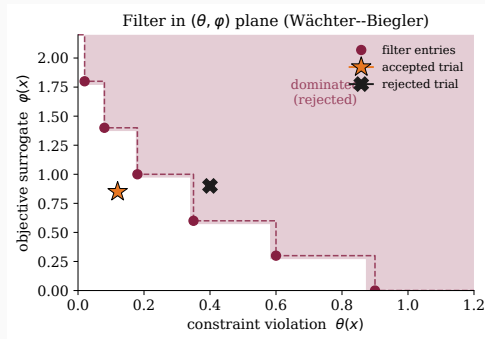
- Barrier objective $\varphi(\mathbf{x}) = f(\mathbf{x}) - \mu \sum \log(-g_i(\mathbf{x}))$
- Equality violation $\theta(\mathbf{x}) = \|\mathbf{h}(\mathbf{x})\|_1$

Filter $\mathcal{F}$. A set of pairs (θ', φ') from past iterates. A new point $\mathbf{x}^+$ is *acceptable to $\mathcal{F}$* if, for every $(\theta', \varphi') \in \mathcal{F}$:

$$\theta(\mathbf{x}^+) \leq (1 - \gamma_\theta)\theta' \quad \text{or} \quad \varphi(\mathbf{x}^+) \leq \varphi' - \gamma_\varphi \theta(\mathbf{x}^+).$$

Reading. Accept if $\mathbf{x}^+$ is strictly better than every filter entry, in *at least one* of the two dimensions (with margin $\gamma_\theta, \gamma_\varphi \approx 10^{-5}$).

No penalty parameter anywhere. The filter is *the* acceptance criterion.



The shaded region is dominated — rejected.
Everything else is acceptable.

Origin. Filter methods for NLP: Fletcher–Leyffer *Math. Prog.* 2002 (SQP setting). Wächter–Biegler *SIAM J. Optim.* 2005 adapted them to IPOPT with the barrier objective φ .

Filter mechanics: when to augment, f-type vs h-type

Step 1 — Compute trial point. $\mathbf{x}^+ = \mathbf{x}^k + \alpha \Delta \mathbf{x}$ starting from $\alpha = \alpha^{\max}$ (fraction-to-boundary).

Step 2 — Check acceptance to filter. If $(\theta(\mathbf{x}^+), \varphi(\mathbf{x}^+))$ is dominated, shrink $\alpha \rightarrow \alpha/2$ and retry.

Step 3 — If accepted, classify the iteration.

- **f-type** iteration: the Newton direction predicts sufficient decrease in φ (switching condition holds), *and* φ actually decreased. Treat as a “normal” optimization step — **do not augment** the filter.
- **h-type** iteration: otherwise. The step reduces θ but not φ . **Augment** $\mathcal{F}$ by adding $(\theta(\mathbf{x}^k), \varphi(\mathbf{x}^k))$.

Why this asymmetry? We want two forms of progress:

- On *good* (f-type) steps: don't clutter the filter — avoid locking ourselves out of this region later.
- On *repair* (h-type) steps: lock out the current (θ, φ) pair to guarantee we don't revisit.

Switching condition (schematic): f-type if the Armijo-type inequality $\varphi(\mathbf{x}^+) \leq \varphi(\mathbf{x}^k) - \eta \alpha \nabla \varphi^\top \Delta \mathbf{x}$ holds AND $\theta(\mathbf{x}^k)$ is small enough that we can trust $\nabla \varphi$ as a descent direction on the nonlinear objective.

Restoration phase and convergence guarantees

What if every $\alpha > 0$ along $\Delta \mathbf{x}$ is rejected by $\mathcal{F}$?

This can happen if the current direction is incompatible with the filter. Wächter–Biegler enter a *restoration phase*: temporarily abandon optimality and chase feasibility alone.

Restoration subproblem. Solve

$$\min_{\mathbf{x}} \|\mathbf{h}(\mathbf{x})\|_2^2 + (\text{regularization})$$

using the same interior-point machinery, until $\theta(\mathbf{x})$ is small enough that the trial point is again acceptable to the filter. Then return to the main IPOPT loop.

Theorem [Wächter–Biegler 2005]. Under standard regularity assumptions (LICQ at limit points, second-order sufficient conditions), the algorithm produces iterates such that *either*

- a subsequence $\{\mathbf{x}^k\}$ converges to a first-order KKT point of the original NLP, or
- the restoration phase detects local infeasibility (no KKT point exists locally).

Why no cycling? Each filter augmentation carves out a neighborhood of (θ', φ') permanently. In finitely many iterations either we accumulate finitely many augmentations (and then stay in an f-type region, where Newton + Armijo works) or we detect infeasibility.

In practice. Restoration fires rarely on well-posed NLPs. When it does, IPOPT reports it in the log (the “R” line prefix). If restoration cannot reduce θ , IPOPT terminates with “Converged to a point of local infeasibility”.

Inertia correction: dealing with nonconvexity

For convex problems, the KKT matrix is positive-definite on the null space of $J_h \Rightarrow$ Newton direction is descent.

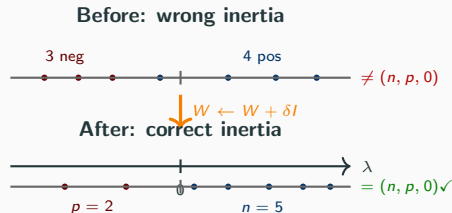
For **nonconvex** problems, W can have wrong-signed eigenvalues, and the Newton direction can fail to be a descent direction.

IPOPT's fix: inertia correction. After factorizing the KKT matrix, check that inertia is $(n_x, n_h, 0)$. If not, add a multiple of the identity to W :

$$W \leftarrow W + \delta I,$$

increasing δ until the inertia is correct. Search over $\delta \in \{10^{-8}, 10^{-4}, 10^{-2}, \dots\}$.

This is *regularization inside the solver*, not a user-facing knob. It's why IPOPT can handle nonconvex NLPs and return local KKT points — though of course not global ones.



Eigenvalues of K . Shift W until sign pattern matches target.

Convex? Nonconvex? IPOPT doesn't care — and that should worry you

IPOPT treats every NLP the same way. The algorithm never asks “is f convex?” or “is $\{g_i \leq 0\}$ a convex set?”. It just:

- Builds the perturbed KKT system.
- Factorizes $K = \begin{pmatrix} W + J_g^T D J_g & J_h^T \\ J_h & 0 \end{pmatrix}$ with inertia correction if needed.
- Takes a Newton step filtered by the filter line search.
- Decreases μ .

This is a *feature* — no solver-selection step, no convexity check — and a *trap*.

What IPOPT actually guarantees (Wächter–Biegler 2005). Under LICQ and second-order sufficient conditions at limit points, a subsequence of iterates converges to a point satisfying the **first-order KKT conditions** of the NLP or the algorithm detects local infeasibility.

What IPOPT does not guarantee:

- Global optimality (obviously, in nonconvex problems).
- That the returned point is a local *minimum* — it could be a saddle point or maximum that happens to satisfy first-order KKT.
- That multi-starts from different $\mathbf{x}^{(0)}$ will agree. On nonconvex problems **they usually don't**.
- That the “objective value” reported is comparable to another run with a different tolerance.

How IPOPT survives nonconvexity:

- **Inertia correction** (previous slide): force K to have correct inertia $(n_x, n_h, 0)$ — makes the direction a descent direction for the filter merit, even if W is not positive definite.
- **Filter line search**: accepts nonmonotone steps, avoids cycling in indefinite regions.
- **Restoration phase**: when no descent direction on the barrier objective exists, chase feasibility alone.
- **Bound relaxation**: $x_L - \epsilon \leq \mathbf{x} \leq x_U + \epsilon$ internally, to avoid artificial ill-conditioning.

Numerical accuracy: when IPOPT “cheats”

IPOPT doesn't return exact solutions. It returns iterates where a *scaled* KKT residual is below a tolerance:

$$\max\left(\frac{\|\nabla_x \mathcal{L}\|_\infty}{s_d}, \|\mathbf{h}\|_\infty, \frac{\|S\Sigma\mathbf{1}\|_\infty}{s_c}\right) \leq \text{tol} \quad (\text{default } 10^{-8}).$$

with auxiliary tolerances `constr_viol_tol`, `dual_inf_tol`, `compl_inf_tol`. The scales s_d, s_c can *inflate* the tolerated residual when multipliers are large.

Daniel Bienstock's favorite example. On certain nonconvex polynomial NLPs, IPOPT reports “Solve.Succeeded” at a point where the *raw* (unscaled) constraint residual $\|\mathbf{h}(\mathbf{x})\|$ is much larger than the user expects — sometimes orders of magnitude larger than `tol`. The solver hasn't lied: by its own scaled metric, it converged. But the returned $\mathbf{x}$ isn't actually a feasible point of the stated problem. If you're using IPOPT to get a *bound* for a MINLP or a polynomial optimization problem, this “solution” can produce a bogus objective value that beats the true global optimum — IPOPT “cheats” by being slightly infeasible.

Trust but verify. After every IPOPT solve, **independently check**:

- $\|\mathbf{h}(\mathbf{x}^*)\|_\infty$ and $\max_i g_i(\mathbf{x}^*)$ computed at full precision — not what the solver reports.
- Do they meet *your* feasibility requirement (not IPOPT's scaled internal one)?
- Is the objective consistent with a neighboring feasible point?
- If multi-starting, how much do solutions disagree?

Mitigations in practice.

- Tighten: `tol=1e-12`, `constr_viol_tol=1e-10`, `bound_relax_factor=0`.
- Scale your problem yourself; don't rely on IPOPT's autoscaling for ill-conditioned constraints.
- For global bounds: use a global solver (BARON, SCIP, Gurobi's nonconvex QP/MIQCP mode). Never quote IPOPT output as a global bound.
- Post-solve projection onto the constraint set if you need strictly feasible iterates.

The IPOPT algorithm, assembled

High-level IPOPT iteration

1. **Initialize** $(\mathbf{x}^0, \mathbf{y}^0, \mathbf{s}^0)$ in the strict interior; μ_0 ; filter $\mathcal{F} = \emptyset$.
2. **Outer loop on μ** : While $\mu > \epsilon$:
 - 2.1 **Inner loop**: while KKT residual at current μ is not small:
 - 2.1.1 Factor the KKT matrix; *inertia correction* if needed.
 - 2.1.2 Solve for $(\Delta \mathbf{x}, \Delta \mathbf{y}, \Delta \mathbf{s})$.
 - 2.1.3 Compute $\alpha^{\max}$ via fraction-to-the-boundary.
 - 2.1.4 *Filter line search*: backtrack from $\alpha^{\max}$ until trial step is filter-acceptable.
 - 2.1.5 Update $(\mathbf{x}, \mathbf{y}, \mathbf{s})$.
 - 2.2 **Decrease μ** (e.g. $\mu \leftarrow \max(0.2\mu, \mu^{1.5})$).

Convergence (convex case): polynomial, $O(\sqrt{n} \log(1/\epsilon))$ outer iterations.

Convergence (nonconvex case): local — to a KKT point under standard assumptions (constraint qualifications).

Roadmap

Motivation: Why KKT is Not an Algorithm

The Logarithmic Barrier

The Central Path

Path-Following and Primal-Dual Interior Point

Coding Break: IPOPT via Pyomo

Convergence Theory: Why It Works

IPOPT: What Makes It Industrial Strength

Wrap-Up

Today's story in three sentences

1. Replace inequalities with a **log barrier** $\Rightarrow$ smooth problem per $\mu > 0$.
2. The solutions trace a **central path** satisfying perturbed KKT: $s_i \cdot (-g_i) = \mu$.
3. Run Newton on the full primal-dual system, drive $\mu \rightarrow 0$, use a filter line search and inertia correction to stay robust. That's IPOPT.

Key complexity fact

For convex problems, interior point methods converge in $O(\sqrt{\nu} \log(1/\epsilon))$ iterations, where ν is the *self-concordance parameter of the barrier* — distinct from the barrier parameter μ that we drive to 0 (coming Thursday).

Thursday preview. The log barrier we used is a special case of a **self-concordant barrier**. Nesterov & Nemirovski (1994) showed that every convex cone admits such a barrier, with a known complexity parameter ν . That's the theory behind CVXPY.

Further reading

- **Nocedal & Wright**, *Numerical Optimization* (2ed), Ch. 19 — the definitive pedagogical treatment.
- **Biegler**, *Nonlinear Programming* (SIAM 2010), Ch. 10–11 — IPOPT insider's view.
- **Wächter**, *Nonlinear Optimization* lecture notes (in the course folder, 4 parts) — by the creator of IPOPT.
- **Wächter & Biegler** (2006), “On the implementation of a primal-dual interior-point filter line-search algorithm for large-scale nonlinear programming,” *Math. Programming* 106(1), 25–57. — **the IPOPT paper**.
- **Boyd & Vandenberghe**, *Convex Optimization*, Ch. 11 — clean convex-only treatment.

Companion notebook: `part1_solvers/04_ipopt_pyomo_cyipopt.ipynb` in the course repo.